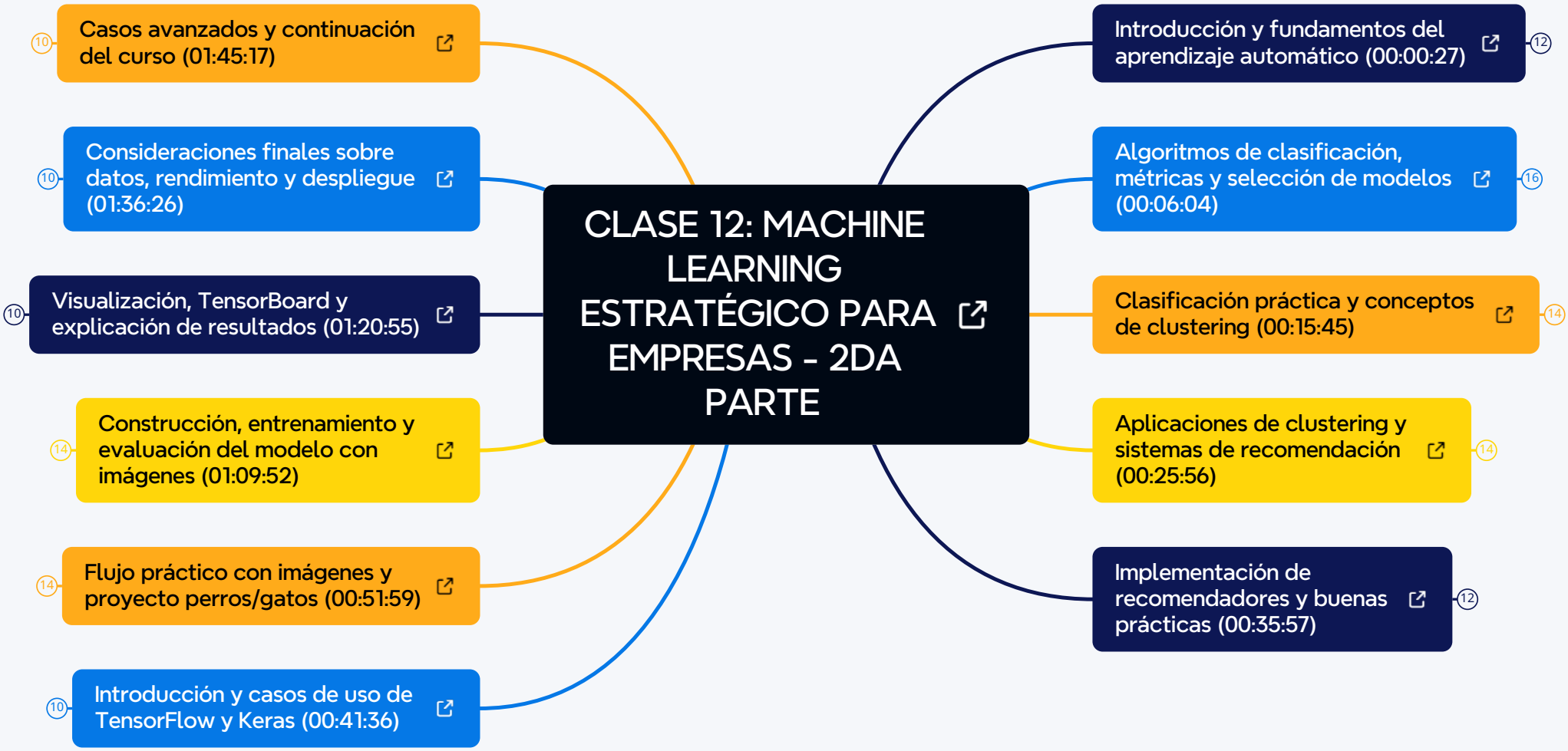
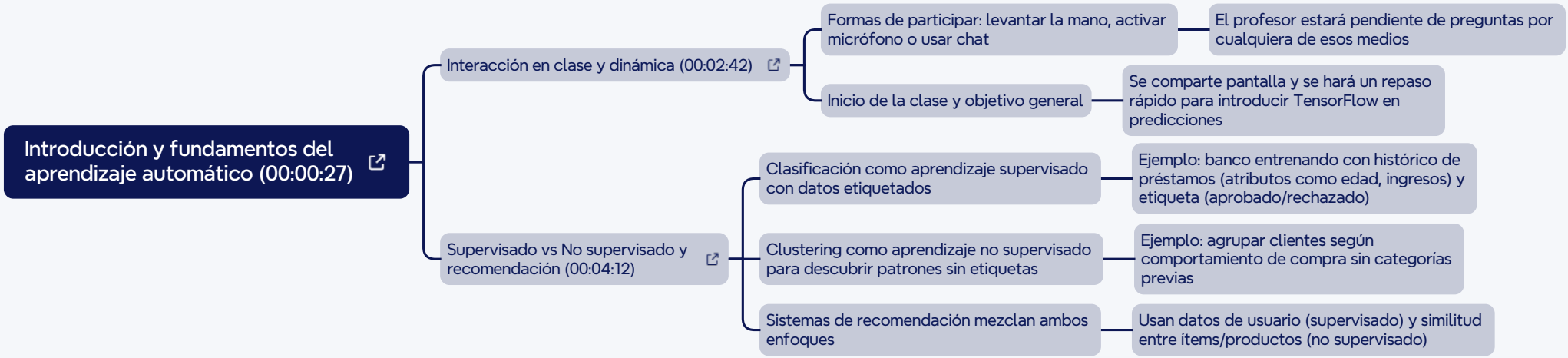
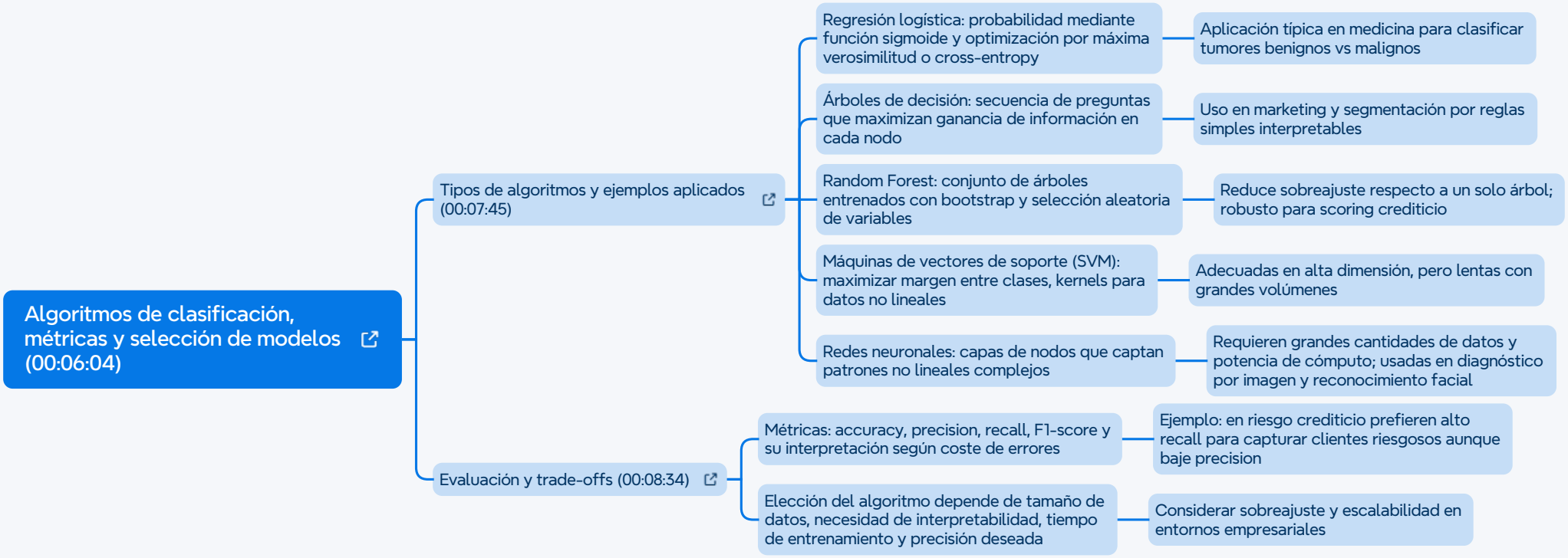
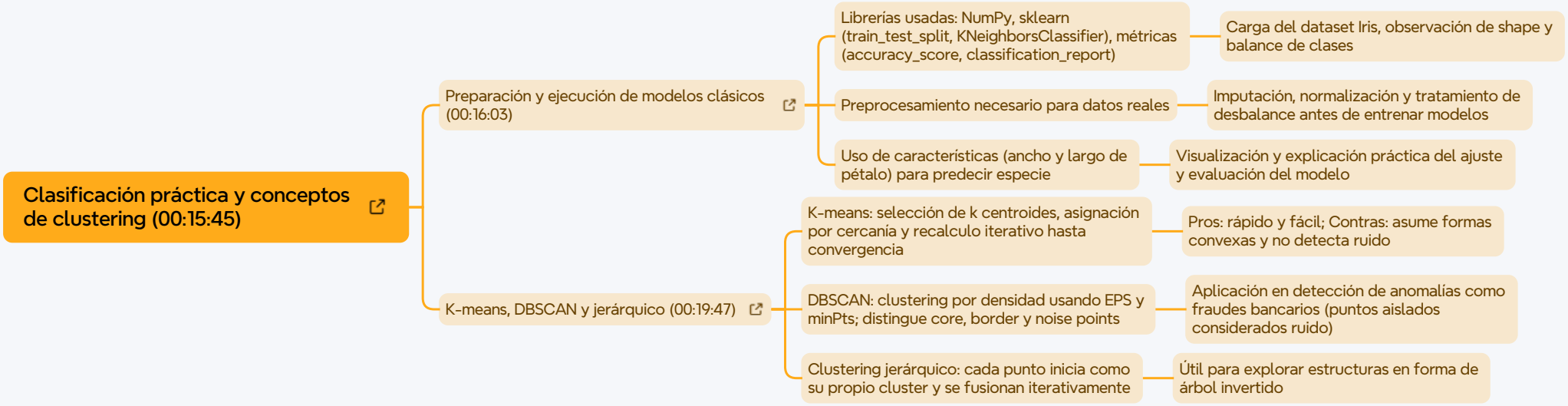
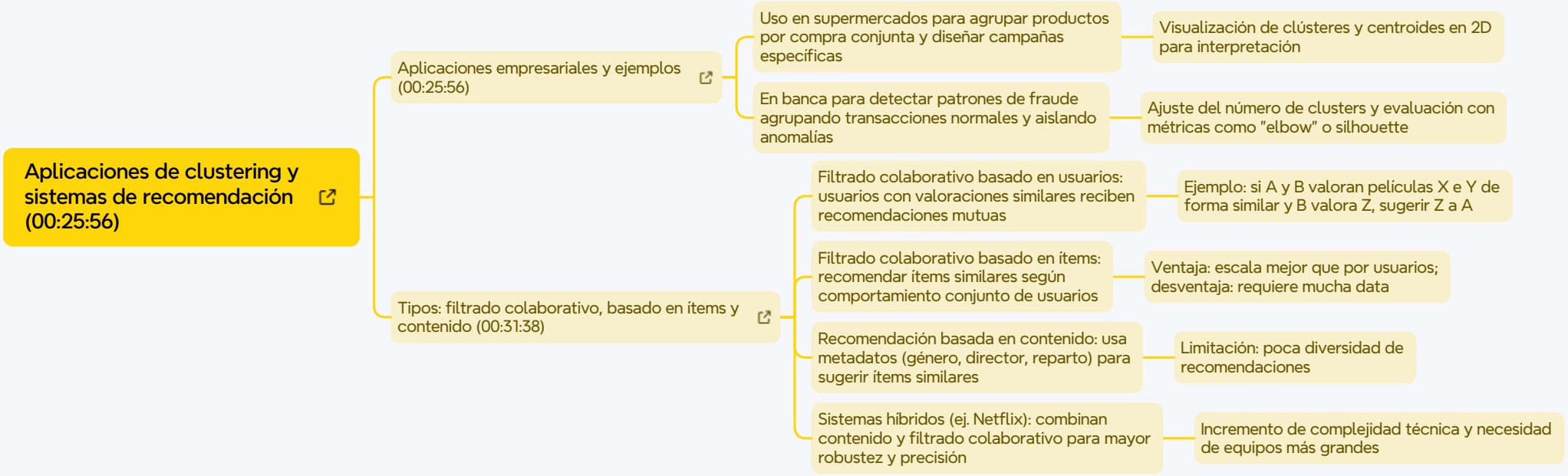
EDU MIN

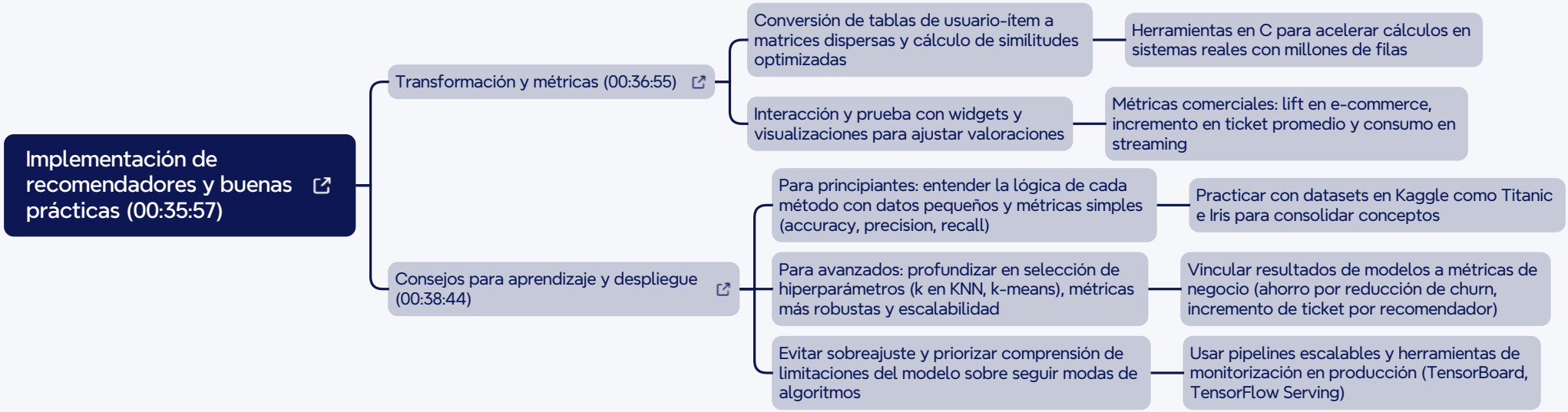

CLASE 12: MACHINE LEARNING ESTRATÉGICO PARA EMPRESAS - 2DA PARTE
Introducción y fundamentos del aprendizaje automático (00:00:27)
Algoritmos de clasificación, métricas y selección de modelos (00:06:04)
Clasificación práctica y conceptos de clustering (00:15:45)
Aplicaciones de clustering y sistemas de recomendación (00:25:56)
Implementación de recomendadores y buenas prácticas (00:35:57)
Casos avanzados y continuación del curso (01:45:17)
Consideraciones finales sobre datos, rendimiento y despliegue (01:36:26)
Visualización, TensorBoard y explicación de resultados (01:20:55)
Construcción, entrenamiento y evaluación del modelo con imágenes (01:09:52)
Flujo práctico con imágenes y proyecto perros/gatos (00:51:59)
Introducción y casos de uso de TensorFlow y Keras (00:41:36)

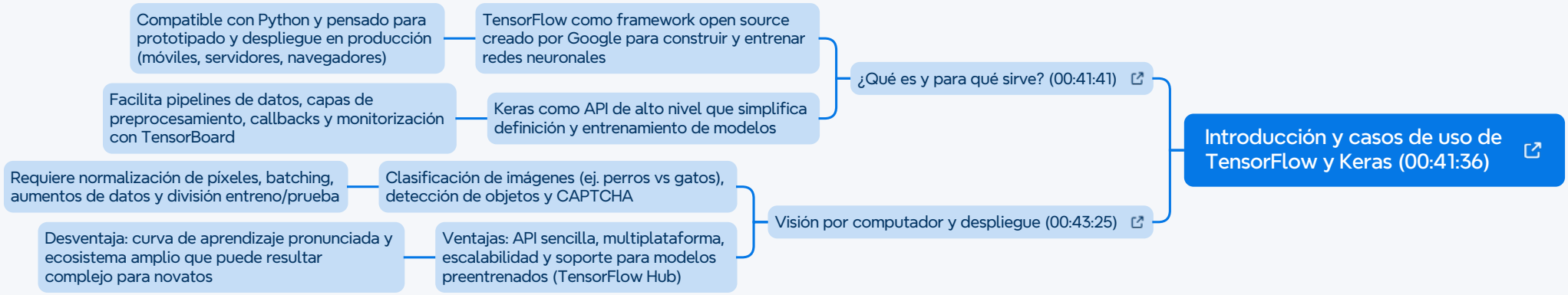

Introducción y fundamentos del aprendizaje automático (00:00:27)
Interacción en clase y dinámica (00:02:42)
Formas de participar: levantar la mano, activar micrófono o usar chat
El profesor estará pendiente de preguntas por cualquiera de esos medios
Inicio de la clase y objetivo general
Se comparte pantalla y se hará un repaso rápido para introducir TensorFlow en predicciones
Supervisado vs No supervisado y recomendación (00:04:12)
Clasificación como aprendizaje supervisado con datos etiquetados
Ejemplo: banco entrenando con histórico de préstamos (atributos como edad, ingresos) y etiqueta (aprobado/rechazado)
Clustering como aprendizaje no supervisado para descubrir patrones sin etiquetas
Ejemplo: agrupar clientes según comportamiento de compra sin categorías previas
Sistemas de recomendación mezclan ambos enfoques
Usan datos de usuario (supervisado) y similitud entre ítems/productos (no supervisado)

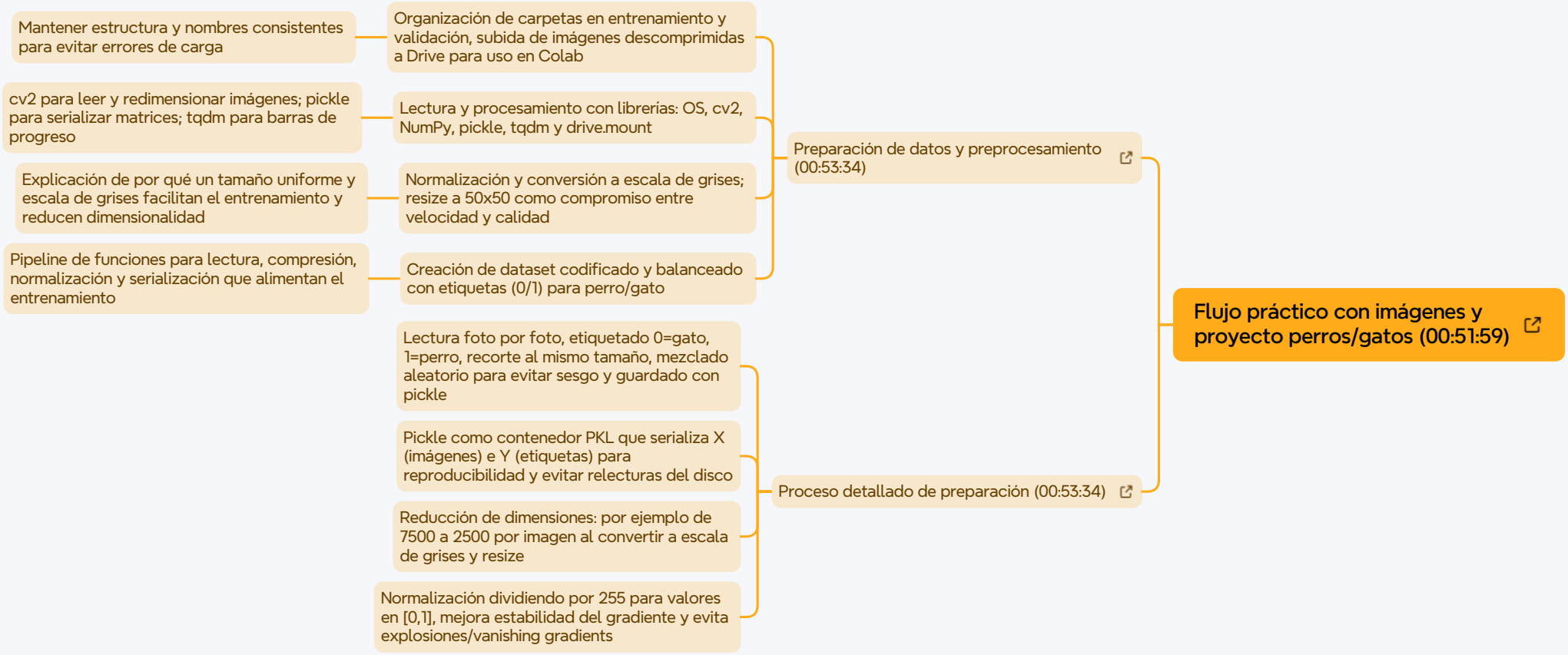

Algoritmos de clasificación, métricas y selección de modelos (00:06:04)
Tipos de algoritmos y ejemplos aplicados (00:07:45)
Regresión logística: probabilidad mediante función sigmoide y optimización por máxima verosimilitud o cross-entropy
Aplicación típica en medicina para clasificar tumores benignos vs malignos
Árboles de decisión: secuencia de preguntas que maximizan ganancia de información en cada nodo
Uso en marketing y segmentación por reglas simples interpretables
Random Forest: conjunto de árboles entrenados con bootstrap y selección aleatoria de variables
Reduce sobreajuste respecto a un solo árbol; robusto para scoring crediticio
Máquinas de vectores de soporte (SVM): maximizar margen entre clases, kernels para datos no lineales
Adecuadas en alta dimensión, pero lentas con grandes volúmenes
Redes neuronales: capas de nodos que captan patrones no lineales complejos
Requieren grandes cantidades de datos y potencia de cómputo; usadas en diagnóstico por imagen y reconocimiento facial
Evaluación y trade-offs (00:08:34)
Métricas: accuracy, precision, recall, F1-score y su interpretación según coste de errores
Ejemplo: en riesgo crediticio prefieren alto recall para capturar clientes riesgosos aunque baje precision
Elección del algoritmo depende de tamaño de datos, necesidad de interpretabilidad, tiempo de entrenamiento y precisión deseada
Considerar sobreajuste y escalabilidad en entornos empresariales

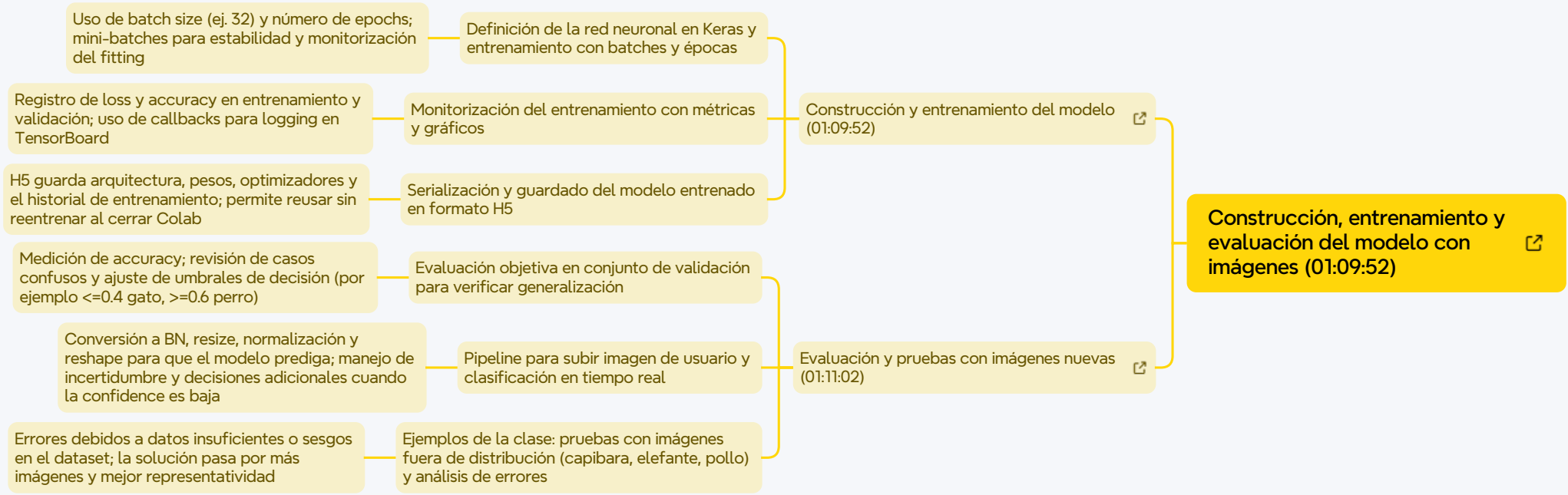

Clasificación práctica y conceptos de clustering (00:15:45)
Preparación y ejecución de modelos clásicos (00:16:03)
Librerías usadas: NumPy, sklearn (train_test_split, KNeighborsClassifier), métricas (accuracy_score, classification_report)
Carga del dataset Iris, observación de shape y balance de clases
Preprocesamiento necesario para datos reales
Imputación, normalización y tratamiento de desbalance antes de entrenar modelos
Uso de características (ancho y largo de pétalo) para predecir especie
Visualización y explicación práctica del ajuste y evaluación del modelo
K-means, DBSCAN y jerárquico (00:19:47)
K-means: selección de k centroides, asignación por cercanía y recalculo iterativo hasta convergencia
Pros: rápido y fácil; Contras: asume formas convexas y no detecta ruido
DBSCAN: clustering por densidad usando EPS y minPts; distingue core, border y noise points
Aplicación en detección de anomalías como fraudes bancarios (puntos aislados considerados ruido)
Clustering jerárquico: cada punto inicia como su propio cluster y se fusionan iterativamente
Útil para explorar estructuras en forma de árbol invertido

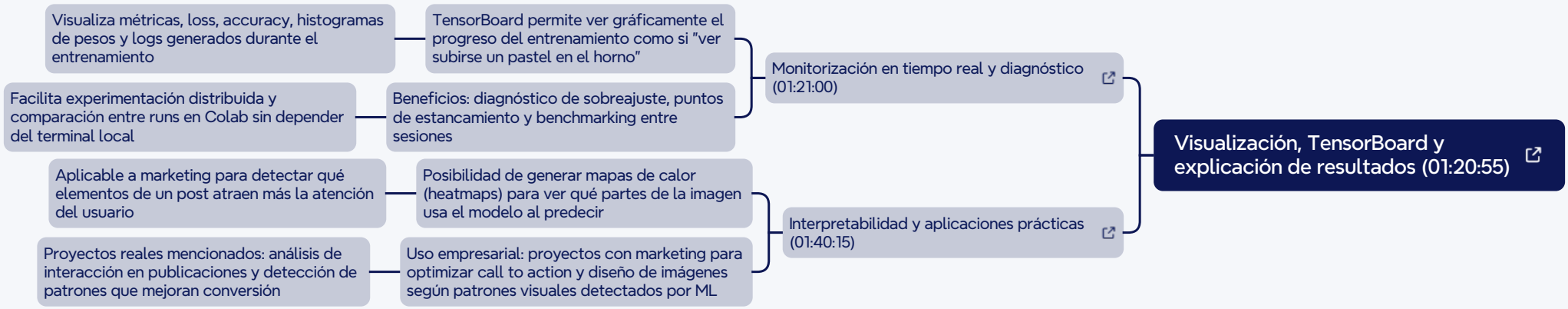

Aplicaciones de clustering y sistemas de recomendación (00:25:56)
Aplicaciones empresariales y ejemplos (00:25:56)
Uso en supermercados para agrupar productos por compra conjunta y diseñar campañas específicas
Visualización de clústeres y centroides en 2D para interpretación
En banca para detectar patrones de fraude agrupando transacciones normales y aislando anomalías
Ajuste del número de clusters y evaluación con métricas como "elbow" o silhouette
Tipos: filtrado colaborativo, basado en ítems y contenido (00:31:38)
Filtrado colaborativo basado en usuarios: usuarios con valoraciones similares reciben recomendaciones mutuas
Ejemplo: si A y B valoran películas X e Y de forma similar y B valora Z, sugerir Z a A
Filtrado colaborativo basado en ítems: recomendar ítems similares según comportamiento conjunto de usuarios
Ventaja: escala mejor que por usuarios; desventaja: requiere mucha data
Recomendación basada en contenido: usa metadatos (género, director, reparto) para sugerir ítems similares
Limitación: poca diversidad de recomendaciones
Sistemas híbridos (ej. Netflix): combinan contenido y filtrado colaborativo para mayor robustez y precisión
Incremento de complejidad técnica y necesidad de equipos más grandes

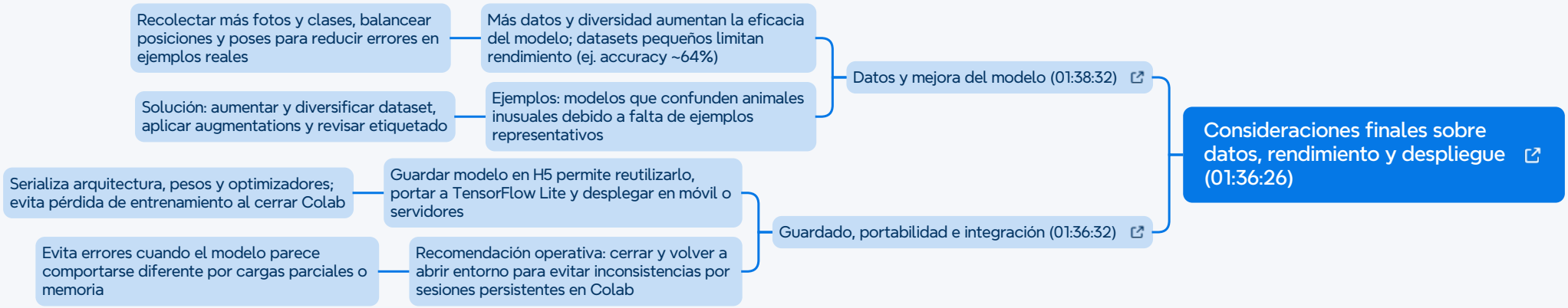

Implementación de recomendadores y buenas prácticas (00:35:57)
Transformación y métricas (00:36:55)
Conversión de tablas de usuario-ítem a matrices dispersas y cálculo de similitudes optimizadas
Herramientas en C para acelerar cálculos en sistemas reales con millones de filas
Interacción y prueba con widgets y visualizaciones para ajustar valoraciones
Métricas comerciales: lift en e-commerce, incremento en ticket promedio y consumo en streaming
Consejos para aprendizaje y despliegue (00:38:44)
Para principiantes: entender la lógica de cada método con datos pequeños y métricas simples (accuracy, precision, recall)
Practicar con datasets en Kaggle como Titanic e Iris para consolidar conceptos
Para avanzados: profundizar en selección de hiperparámetros (k en KNN, k-means), métricas más robustas y escalabilidad
Vincular resultados de modelos a métricas de negocio (ahorro por reducción de churn, incremento de ticket por recomendador)
Evitar sobreajuste y priorizar comprensión de limitaciones del modelo sobre seguir modas de algoritmos
Usar pipelines escalables y herramientas de monitorización en producción (TensorBoard, TensorFlow Serving)

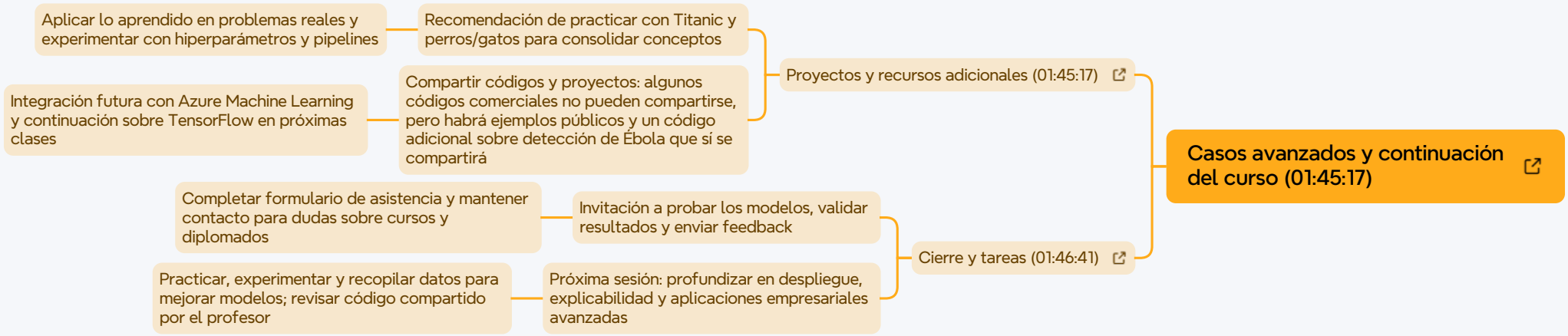

Introducción y casos de uso de TensorFlow y Keras (00:41:36)
¿Qué es y para qué sirve? (00:41:41)
TensorFlow como framework open source creado por Google para construir y entrenar redes neuronales
Compatible con Python y pensado para prototipado y despliegue en producción (móviles, servidores, navegadores)
Keras como API de alto nivel que simplifica definición y entrenamiento de modelos
Facilita pipelines de datos, capas de preprocesamiento, callbacks y monitorización con TensorBoard
Visión por computador y despliegue (00:43:25)
Clasificación de imágenes (ej. perros vs gatos), detección de objetos y CAPTCHA
Requiere normalización de píxeles, batching, aumentos de datos y división entreno/prueba
Ventajas: API sencilla, multiplataforma, escalabilidad y soporte para modelos preentrenados (TensorFlow Hub)
Desventaja: curva de aprendizaje pronunciada y ecosistema amplio que puede resultar complejo para novatos


Flujo práctico con imágenes y proyecto perros/gatos (00:51:59)
Preparación de datos y preprocesamiento (00:53:34)
Organización de carpetas en entrenamiento y validación, subida de imágenes descomprimidas a Drive para uso en Colab
Mantener estructura y nombres consistentes para evitar errores de carga
Lectura y procesamiento con librerías: OS, cv2, NumPy, pickle, tqdm y drive.mount
cv2 para leer y redimensionar imágenes; pickle para serializar matrices; tqdm para barras de progreso
Normalización y conversión a escala de grises; resize a 50x50 como compromiso entre velocidad y calidad
Explicación de por qué un tamaño uniforme y escala de grises facilitan el entrenamiento y reducen dimensionalidad
Creación de dataset codificado y balanceado con etiquetas (0/1) para perro/gato
Pipeline de funciones para lectura, compresión, normalización y serialización que alimentan el entrenamiento
Proceso detallado de preparación (00:53:34)
Lectura foto por foto, etiquetado 0=gato, 1=perro, recorte al mismo tamaño, mezclado aleatorio para evitar sesgo y guardado con pickle
Pickle como contenedor PKL que serializa X (imágenes) e Y (etiquetas) para reproducibilidad y evitar relecturas del disco
Reducción de dimensiones: por ejemplo de 7500 a 2500 por imagen al convertir a escala de grises y resize
Normalización dividiendo por 255 para valores en [0,1], mejora estabilidad del gradiente y evita explosiones/vanishing gradients


Construcción, entrenamiento y evaluación del modelo con imágenes (01:09:52)
Construcción y entrenamiento del modelo (01:09:52)
Definición de la red neuronal en Keras y entrenamiento con batches y épocas
Uso de batch size (ej. 32) y número de epochs; mini-batches para estabilidad y monitorización del fitting
Monitorización del entrenamiento con métricas y gráficos
Registro de loss y accuracy en entrenamiento y validación; uso de callbacks para logging en TensorBoard
Serialización y guardado del modelo entrenado en formato H5
H5 guarda arquitectura, pesos, optimizadores y el historial de entrenamiento; permite reusar sin reentrenar al cerrar Colab
Evaluación y pruebas con imágenes nuevas (01:11:02)
Evaluación objetiva en conjunto de validación para verificar generalización
Medición de accuracy; revisión de casos confusos y ajuste de umbrales de decisión (por ejemplo <=0.4 gato, >=0.6 perro)
Pipeline para subir imagen de usuario y clasificación en tiempo real
Conversión a BN, resize, normalización y reshape para que el modelo prediga; manejo de incertidumbre y decisiones adicionales cuando la confidence es baja
Ejemplos de la clase: pruebas con imágenes fuera de distribución (capibara, elefante, pollo) y análisis de errores
Errores debidos a datos insuficientes o sesgos en el dataset; la solución pasa por más imágenes y mejor representatividad


Visualización, TensorBoard y explicación de resultados (01:20:55)
Monitorización en tiempo real y diagnóstico (01:21:00)
TensorBoard permite ver gráficamente el progreso del entrenamiento como si "ver subirse un pastel en el horno"
Visualiza métricas, loss, accuracy, histogramas de pesos y logs generados durante el entrenamiento
Beneficios: diagnóstico de sobreajuste, puntos de estancamiento y benchmarking entre sesiones
Facilita experimentación distribuida y comparación entre runs en Colab sin depender del terminal local
Interpretabilidad y aplicaciones prácticas (01:40:15)
Posibilidad de generar mapas de calor (heatmaps) para ver qué partes de la imagen usa el modelo al predecir
Aplicable a marketing para detectar qué elementos de un post atraen más la atención del usuario
Uso empresarial: proyectos con marketing para optimizar call to action y diseño de imágenes según patrones visuales detectados por ML
Proyectos reales mencionados: análisis de interacción en publicaciones y detección de patrones que mejoran conversión


Consideraciones finales sobre datos, rendimiento y despliegue (01:36:26)
Datos y mejora del modelo (01:38:32)
Más datos y diversidad aumentan la eficacia del modelo; datasets pequeños limitan rendimiento (ej. accuracy ~64%)
Recolectar más fotos y clases, balancear posiciones y poses para reducir errores en ejemplos reales
Ejemplos: modelos que confunden animales inusuales debido a falta de ejemplos representativos
Solución: aumentar y diversificar dataset, aplicar augmentations y revisar etiquetado
Guardado, portabilidad e integración (01:36:32)
Guardar modelo en H5 permite reutilizarlo, portar a TensorFlow Lite y desplegar en móvil o servidores
Serializa arquitectura, pesos y optimizadores; evita pérdida de entrenamiento al cerrar Colab
Recomendación operativa: cerrar y volver a abrir entorno para evitar inconsistencias por sesiones persistentes en Colab
Evita errores cuando el modelo parece comportarse diferente por cargas parciales o memoria


Casos avanzados y continuación del curso (01:45:17)
Proyectos y recursos adicionales (01:45:17)
Recomendación de practicar con Titanic y perros/gatos para consolidar conceptos
Aplicar lo aprendido en problemas reales y experimentar con hiperparámetros y pipelines
Compartir códigos y proyectos: algunos códigos comerciales no pueden compartirse, pero habrá ejemplos públicos y un código adicional sobre detección de Ébola que sí se compartirá
Integración futura con Azure Machine Learning y continuación sobre TensorFlow en próximas clases
Cierre y tareas (01:46:41)
Invitación a probar los modelos, validar resultados y enviar feedback
Completar formulario de asistencia y mantener contacto para dudas sobre cursos y diplomados
Próxima sesión: profundizar en despliegue, explicabilidad y aplicaciones empresariales avanzadas
Practicar, experimentar y recopilar datos para mejorar modelos; revisar código compartido por el profesor